

velocoach-rs: A Zero-Cost, Sensor-Free Demo Plan for Your Rust Trial

Verdict: build it the way you proposed, with one change. Make the iQOO an Android GATT peripheral that simulates the sensors, make the iPhone 12 a CoreBluetooth central running the Rust core through UniFFI, and use the Mac for builds, a btleplug desktop harness and FIT replay. Every part of that works with free tools and a free Apple ID. The change: the simulator's own Kotlin code should *not* run the physiology. The Rust `(vc-sim)` crate should own the rider model and the packet encoders, so Android is a thin radio shell, just like iOS. That way one Rust codebase both creates the bytes on one phone and parses them on the other. The demo story is: "One Rust core, two mobile platforms, a real 2.4 GHz BLE link, standard SIG characteristics, and a simulated rider who can be made to fail on demand. Each fault (W' exhaustion, HR redline, a dead sensor, coasting) produces a specific, tested, latency-measured voice cue."

TL;DR

- **The setup works and costs nothing.** Android apps can advertise and serve standard GATT services after getting the `(BLUETOOTH_ADVERTISE)`/`(BLUETOOTH_CONNECT)` runtime permissions. A `(connectedDevice)` foreground service keeps the iQOO alive. A free Apple ID lets you run your own app on the iPhone 12, but the profile expires every 7 days and you get 3 apps per device. iOS 27 (released 14 Sep 2026) still supports the iPhone 12. Fallbacks: nRF Connect for Android can act as a no-code HR, power or CSC simulator, and FIT replay on the Mac needs no radio at all.
- **Architecture: a sans-IO Rust workspace shared by both phones.** `(vc-gatt)` (encode and decode), `(vc-sim)` (rider model), `(vc-physio)`, `(vc-core)` and `(vc-ffi)` run on both phones. A custom "sim-telemetry" characteristic carries sequence numbers and the simulator's send timestamps, so you can measure sensor-to-decision latency over the real radio. Report audio honestly: on iOS, the A2DP output latency (`(outputLatency)`) is typically about 160 ms by itself.
- **Schedule: the full end-to-end slice runs by day 11. Days 12–20 harden it, measure it, script fault scenarios, and record a 3-minute video.** The first cuts if you fall behind are FTMS, the Android-side TTS, and the Intervals.icu write path.

Key Findings (verified against sources, with uncertainty flagged)

1. Android as a BLE peripheral (iQOO)

- **Permissions.** `(BluetoothLeAdvertiser.startAdvertising)` always enforces `(BLUETOOTH_ADVERTISE)`. For apps targeting Android 12 (API 31) or higher, it is a runtime permission requested through `(requestPermissions)`. `(android)` `(BLUETOOTH_ADVERTISE)`, `(BLUETOOTH_CONNECT)` and `(BLUETOOTH_SCAN)` are all runtime permissions, and the legacy `(BLUETOOTH)`/`(BLUETOOTH_ADMIN)` should carry

`android:maxSdkVersion="30"`. `android` `devsflow` The simulator never scans, so it doesn't need `BLUETOOTH_SCAN` or location.

- **Check for peripheral support before advertising.** `getBluetoothLeAdvertiser()` "will return null if Bluetooth is turned off or if Bluetooth LE Advertising is not supported". Check `isMultipleAdvertisementSupported()` first. `android` `aiden` Any recent iQOO almost certainly supports it, but check on day 1 anyway.
- **Foreground service.** Apps targeting Android 14 must declare a foreground-service type. For BLE it is `connectedDevice`: declare it in the manifest, add the `FOREGROUND_SERVICE_CONNECTED_DEVICE` permission, and pass `FOREGROUND_SERVICE_TYPE_CONNECTED_DEVICE` to `startForeground()`. `android` `android` The runtime prerequisites include holding a Bluetooth permission such as `BLUETOOTH_ADVERTISE`/`CONNECT`. Without a declared type, `startForeground()` throws `MissingForegroundServiceTypeException`. `medium` Android 13+ also needs `POST_NOTIFICATIONS` to show the service notification. `dev`
- **Vivo/iQOO background killing is real and only partly solved.** Dontkillmyapp.com's Vivo page says "System restrictions on Vivo phones have not been fully uncovered yet," and lists "No known solution on dev end yet" for developers. It recommends Autostart, "High background power consumption" / "Background power consumption management", and locking the app in the recents tray. A 2026 report from an OriginOS 6 user (Android 16) found reliable behavior only with the app permanently locked in the background. `github` **Practical answer: keep the simulator app in the foreground with the screen on for every demo run. Use the FGS plus every Vivo setting for robustness, not as the main defense.**
- **Advertising payload.** Legacy advertising has 31 bytes. Flags take 3 bytes and a 16-bit UUID list costs $2 + 2 \cdot n$, so `0x180D` + `0x1818` + `0x1816` is 8 bytes. Put the service UUIDs in the advertisement and the device name (`VC-SIM`) in the scan response.
- **MTU.** Android's default ATT MTU is 23, which leaves 20 bytes of payload. `devsflow` HRM (2-4 bytes), CPM with crank data (8 bytes) and CSC (11 bytes) all fit without MTU negotiation.
- **No-code fallback.** nRF Connect for Android has a "Configure GATT server" page with a built-in Heart Rate service that "will send sample HRM data when the remote device enable notifications". `github` The open-source Auuki project ships importable nRF Connect GATT-server configs for a Tacx Flux (FTMS), a Heart Rate Monitor, a Speed and Cadence sensor and a Power Meter, plus data macros; its author notes that "only the Android version of this app will work." `github` That is your day-1 radio smoke test and your plan C. "BLE Advertiser — GATT Simulator" is promoted mainly by its own developer's blog, so treat it as unvetted.

2. iOS as the central (iPhone 12), and why that direction is the right one

- **Android peripheral → iOS central is the reliable direction.** The CoreBluetooth docs describe advertising a Heart Rate service via `CBPeripheralManager`, and a 2026

project re-broadcasts HR on 0x180D from an iPhone. (apple +2) But iOS blocks some SIG UUIDs ("The specified UUID is not allowed for this operation" for HID 0x1812), and backgrounded iOS peripherals drop service UUIDs from advertisements. (github) (apple) Use Android as the peripheral.

- **GATT caching and Service Changed.** iOS doesn't cache the attribute table when the peripheral exposes the Generic Attribute service, and it rediscovers on a Service Changed indication. (punchthrough) One documented iOS failure: when an Android (Bluedroid) server removes and then re-adds services, iOS honors the removal but ignores the re-add, so it thinks the peripheral has no services. (appspot) **Rule: register all services once, wait for each (onServiceAdded), then start advertising, and never touch the service table while connected.** After an adapter power-cycle, rebuild the (BluetoothGattServer); the old handle goes stale. (github)
- **Mac and iPhone at the same time.** (notifyCharacteristicChanged(device, ...)) takes a per-device argument, so two centrals is plausible, but I found no source confirming it on Vivo's stack. **Plan for one central at a time**, and treat dual connection as a stretch test.

3. Free Apple ID in October 2026

- **Apple's own limits.** Apple's "Developer account overview" page says "You can register up to 10 App IDs, which expire after 7 days," plus up to 3 devices (also 7 days) and up to 3 apps per device, and provisioning profiles "will expire 7 days from issuance." Quinn "The Eskimo!" of Apple DTS advises that "one easy way to avoid hitting this limit is to use the same bundle ID, and hence the same App ID, for all your programs." iOS 16+ requires Developer Mode before a development-signed build will install. (betadrop)
- **Capabilities.** Background Modes ((bluetooth-central), (audio)) are Info.plist keys, not paid entitlements. The commonly reported blocked list for personal teams is HealthKit, push notifications, iCloud, Sign in with Apple, IAP and similar. (appbuilder24) (builds) I found no source saying CoreBluetooth or the background modes are blocked, so this is high confidence but not documented by Apple. **Verify on day 1** with a 30-line scanner app.
- **iOS and Xcode versions.** iOS 27 shipped on 14 Sep 2026 alongside Xcode 27 and supports the iPhone 11 and later, so the iPhone 12 gets it, though without Apple Intelligence. (igeeksblog) (dev) Xcode 26.4 already required macOS Tahoe 26.2 or later; (apple) expect Xcode 27 to need a current macOS, and check the Xcode system-requirements page before downloading. If your iPhone is on iOS 27, you need Xcode 27 to deploy to it.
- **8 GB RAM.** Xcode works on 8 GB. Just don't run Xcode, a simulator, Android Studio and Chrome at once. Tips are in section (e).

4. Rust toolchain

- **UniFFI.** v0.32.0 shipped 2026-06-30, v0.32.1 on 2026-09-08 ("Kotlin: Fixed checksum failure on aarch64"), and v0.32.2 on 2026-09-22. (github +2) **Pin 0.32.2.**

- Proc-macros require library mode: `(uniffi-bindgen generate --library <cdylib> --language kotlin|swift)`. [github](#) [github](#) `(uniffi-bindgen-swift` always runs in library mode and can emit headers, modulemaps and sources separately. [github](#)
- The 0.32 breaking change: `[ByRef] bytes` now cross zero-copy, and Kotlin call sites must pass a *direct* `java.nio.ByteBuffer`, while Swift `Data` is unchanged. [github](#) [github](#) In proc-macro terms, `fn ingest(&self, ..., payload: &[u8])` takes `ByteBuffer.allocateDirect(n)` from Kotlin.
- The `--config` flag now expects a global config file. [github](#)
- Kotlin needs JNA 5.12.0 or later as an `@aar` `(net.java.dev.jna:jna:5.12.0@aar)`. [github](#) [uniffi.toml](#) `android = true` enables Android-specific optimizations. [github](#) R8 release builds can break JNA without keep rules, [gobley](#) so build debug for the demo.
- **Android without Android Studio.**
 - `brew install --cask android-commandlinetools`, which needs a JDK. The SDK root is `($HOMEBREW_PREFIX/share/android-commandlinetools)`. [github](#)
 - `sdkmanager --install "ndk;30.0.16248370"`. NDK r30 is the current LTS; r29 has moved to "unsupported". [github](#) [android](#)
 - cargo-ndk 4.1.2 is the current release (August 2025, no release since). `cargo ndk -t arm64-v8a -o app/src/main/jniLibs build --release`. [socket +2](#)
 - Set `(ANDROID_NDK_HOME)`, because cargo-ndk only auto-detects Android Studio's default NDK location. [docs](#)

5. Free ride data and W'bal references

- **GoldenCheetah OpenData** is on OSF (osf.io/6hfpz, DOI 10.17605/OSF.IO/6HFPZ) and a public AWS S3 bucket ([goldencheetah-opendata](https://goldencheetah-opendata.s3.amazonaws.com)). Athletes opted in to anonymized sharing; Mark Liversedge's November 2018 blog post reported "over 1300 users have said 'Yes' and shared over 700,000 workouts," he later cited "2.5million workouts and over 6000 athletes," and the GoldenCheetah/OpenData README now says the S3 bucket "is currently closed for public access," so download from OSF. Each athlete's archive contains JSON metadata plus a CSV per activity, not FIT, [trainingwithdata](#) so write a small CSV→[Sample](#) adapter for replay. The [goldencheetah-opendata](#) Python client is MIT-licensed. [libraries](#) **I couldn't confirm the data licence on the OSF project page. Check it before redistributing any file, and ship download scripts rather than data.**
- **FIT files.** Use the fitparser crate's test fixtures and the Garmin FIT SDK example files for parser tests (check each licence). Best of all: use [vc-sim](#) to generate traces that you fully own.
- **W'bal validation.** GoldenCheetah, which is GPL-2.0, [github](#) uses the differential algorithm (Froncioni/Skiba/Clarke) by default, with an implicit time constant $\tau =$

W'/CP. (blogspot) (gssns) Import your simulated ride into GoldenCheetah and export the W'bal series as CSV; compare numbers, don't copy GPL code. Also add analytic test vectors: constant power above CP must deplete linearly at $(P - CP)$ J/s, and recovery at $P=0$ must follow $dW = CP \cdot (W' - W_{bal}) / W'$ per second. (gssns)

6. Rider physiology: credible, simple, cited

- **HR kinetics.** Hunt and Wang (Bern University of Applied Sciences, F1000Research 13:894, v2, 1 Nov 2024), analysing 54 cycle-ergometer data sets from 27 participants, found "a linear first-order system with dead time as being an appropriate model for heart rate response to exercise"; a search-indexed abstract also reports a cycle-ergometer gain of about 0.40 bpm/W (confirm in the full text before quoting). A field-data model uses a 15 s HR lag plus a cardiac-drift term. (arxiv) A cyclist-simulation paper models drift by inflating "perceived power" in proportion to cumulative energy expenditure: $P' = P + k_f \cdot \Sigma P$. (arxiv) **Model:** $HR_{ss} = HR_{rest} + k \cdot P_{eff}$, where $P_{eff} = P(t - 10 \dots 15 s) \cdot (1 + k_f \cdot E_{kJ})$. Then $dHR/dt = (HR_{ss} - HR) / \tau$, with $\tau_{on} \approx 30-40$ s, a slower $\tau_{off} \approx 60$ s, and a clamp at HR_{max} . Cardiac drift is usually defined as $HR(\text{second half})$ vs $HR(\text{first half})$ at constant power, so your Pa:HR decoupling metric has a known ground truth in the simulator, which makes a good test. (nih)
- **W' consistency.** The simulator keeps its own "true" W' state, with parameters you can deliberately mismatch from the coach's estimate. When true $W' \leq 0$, achievable power is capped at $CP \cdot (0.9-1.0)$ and cadence sags. That is the "blow-up."
- **Crank events.** Integrate revolutions as $\text{cadence} / 60 \cdot dt$. On each whole revolution, increment the uint16 cumulative count and set the last event time to $(t_{event} \cdot 1024) \bmod 65536$. When coasting, keep sending the same count and event time (that's how real meters look stale), and in "silent coasting" mode stop notifying entirely.
- **Noise and dropout.** Power gets Gaussian noise (σ 2-4%) plus occasional spikes. HR gets ± 1 bpm quantization. Dropouts are scripted: N-second silences, disconnects, and garbage payloads such as truncated, wrong-flag and all-0xFF packets. Use a seeded RNG ((rand_pcg)) for determinism.

7. Zero-cost TTS

- **Piper.** (pip install piper-tts) works on Apple Silicon. (qwe) (python -m piper.download_voices en_US-ljspeech-medium), then (python -m piper -m en_US-ljspeech-medium.onnx -f out.wav -- "text"). (openspeech)
 - The original rhasspy/piper (MIT) was archived in October 2025; the maintained OHF-Voice/piper1-gpl engine is GPL-3.0. (qwe) (localaimaster)
 - Voices have their own licences: Piper says they're for personal use and research, so check each voice's MODEL_CARD. The LJSpeech voice comes from a public-domain dataset, the cleanest choice for a demo. (cekura) (openspeech)
 - You only *run* the engine offline to make WAV files and never ship it, so GPL obligations don't attach to your app.

- **Alternatives.** iOS (`AVSpeechSynthesizer`) serves as the runtime fallback for phrases not in the bank.
- **Measuring audio latency.**
 - On an Apple Community thread (iPhone 13 + AirPods Pro), a developer measured (`outputLatency`) at "A2DP: ~163ms / HFP: ~9ms" but "in practice, we observed only around a 30ms difference." On Apple Developer Forums thread 679274 (iPhone 12 mini + AirPods 2, (`outputLatency`) 160.667 ms), measured latency started at "ca 215-220ms" and settled "around 155-160ms" after 20-30 minutes (starting near 180 ms only if the AirPods were already in use). Another developer saw 193-260 ms within a minute. (`apple`)
 - So report T3 (`AVAudioPlayerNode` `schedule/()` call) as measured, and T4 = T3 + (`outputLatency`) explicitly labeled as *an OS estimate*.
 - Don't use the HFP route: it cuts latency, but drops audio quality and changes the session category.

8. Intervals.icu

- Generate a personal API key under Settings → Developer Settings. It uses basic auth with username (`API_KEY`) and your key as the password. (`intervals`) Athlete ID (`0`) means "me". (`intervals`)
- (`GET /api/v1/athlete/0/events?category=WORKOUT&ext=zwo&resolve=true&oldest=...&newest=...`) returns (`workout_file_base64`) with the workout converted to .zwo. (`intervals`) Per-event download is (`GET /api/v1/athlete/{id}/events/{eventId}/download.zwo`). (`intervals`)
- Create demo workouts in the web builder ("3×4m VO2") and pull them as .zwo; (`vc-sync`) parses .zwo into a (`WorkoutPlan`). Check that the API key appears on your free account on day 2.

9. Video at zero cost

- iPhone's built-in Screen Recording captures app audio, but connected Bluetooth audio devices can stop it from being captured. (`apple`) (`movavi`) **Record the cue segments with audio routed to the iPhone speaker**, and state on screen that in real use cues play through open-ear headphones.
- Mac: QuickTime/⌘ ↑ 5 for the terminal. iMovie (free) for compositing: phone recording on the left, terminal/latency on the right.

10. Is simulation impressive enough?

Yes, if you frame it as **hardware-in-the-loop over a real radio**, not as a mock. Credibility comes from deterministic seeds ("`vc-cli scenario vo2_blowup --seed 42`) gives byte-identical cue logs"), golden-file tests of cue logs, fault injection that is impossible to do safely on a real ride, latency measured at segment boundaries, and fidelity: the simulator

emits exact SIG byte formats and the decoder is property-tested against the encoder. Skip the walking segment; with no sensors it adds nothing.

Details

(b) System design

Device roles and data flow



Threading model. The Rust core is sans-IO and synchronous: no threads and no allocation in the hot path. `Engine` wraps `Mutex<CoreState>` (UniFFI Objects must be `Send + Sync`) and exposes `ingest(t_ns, src, &[u8])` (parse and update physio, $O(1)$), `tick(t_ns) -> Vec<Cue>` (timeline, gate, arbiter) and `snapshot()` (copy-out for the UI). On Swift, `ingest()` and a `DispatchSourceTimer` calling `tick()` both run on the `ble.serial` queue, so the mutex is uncontended. The audio thread belongs to AVAAudioEngine (you only schedule buffers), and the main actor pulls `snapshot()` at 4 Hz, never per packet. On

Kotlin, a "sim-tick" `HandlerThread` runs `Simulator.step()` and the notify queue. Binder-thread GATT callbacks post to that loop, and the main thread does UI only.

Latency points and clock sync

```
T0  sim_ns stamped by vc-sim just before notifyCharacteristicChanged (iQ00 clock)
T1  iOS didUpdateValueFor entry, mach_continuous_time→ns (iPhone clock)
T2  engine.tick() returns cue (Rust, iPhone clock)
T3  playerNode.scheduleBuffer/play called (iPhone clock)
T4  T3 + AVAudioSession.outputLatency (OS ESTIMATE, labelled as such)

Clock offset (iQ00→iPhone): NTP-style over BLE every 10 s
  iPhone writes CTRL{PING, t_a} → iQ00 notifies TELEM{PONG, t_a, t_s}
  iPhone receives at t_b: rtt = t_b - t_a; offset ≈ t_s - (t_a + rtt/2)
  keep min-RTT sample of last 8 (Cristian's algorithm); error bound = rtt_min/2
Matching: TELEM carries (seq, char_id, sim_ns) sent right after each data notify;
  core matches by seq → T0. Report T0→T1 with ±rtt_min/2 bound.
```

Error handling and fault injection. Every parser returns `Result<_, ParseError>` and never panics; each error type increments a counter. Each sensor channel has a quality state machine: `Fresh → Stale (>3 s) → Lost (>10 s) → Recovering (3 good samples)`. The arbiter drops any cue whose input is Stale. The SafetyGate pre-empts all other cues: $HR \geq HR_{max} - 3$ for 5 s, or an implausible jump ($HR \Delta > 30 \text{ bpm/s}$). Faults come from the `CTRL` characteristic or from scenario scripts in `vc-sim`: `DropChannel(hr, 20s)`, `Garbage(cpm, n)`, `Disconnect(5s)`, `Coast(silent|stale, 15s)`, `WPrimeMismatch(+30%)`, `HrRedline`.

(c) Repo layout

```
velocoach-rs/  
├─ Cargo.toml                # workspace, resolver=2, uniffi = "=0.32.2"  
├─ crates/  
│   ├─ vc-gatt/              #![no_std] decode+encode HRM/CPM/CSC; proptest, cargo-  
fuzz [shared: iOS+Android+Mac]  
│   ├─ vc-physio/            #![no_std] W'bal (diff + Waterworth), decoupling, quality  
[iOS+Mac]  
│   ├─ vc-sim/               rider model, scenarios, seeded RNG; uses vc-gatt encoders  
[Android+Mac]  
│   ├─ vc-core/              Engine: timeline, SafetyGate, Arbiter, cue rules, latency  
ring [iOS+Mac]  
│   ├─ vc-sync/              .zwo parser → WorkoutPlan; Intervals.icu client (std,  
ureq) [Mac; iOS stretch]  
│   └─ vc-ffi/               UniFFI facade: Engine + Simulator objects;  
cdylib+staticlib [both]  
├─ tools/  
│   ├─ vc-cli/               replay | live (btleplug) | scenario | bench ; golden cue  
logs  
│   └─ phrasegen/            phrases.toml → Piper WAV bank + manifest.json (syllable  
check)  
├─ apps/  
│   ├─ ios/VeloCoach/        SwiftUI + CoreBluetooth; Rust via  
VcCore.xcframework (SPM local pkg)  
│   └─ android/vcsim/         Kotlin GATT server; jniLibs/arm64-  
v8a/libvc_ffi.so; JNA aar  
├─ scenarios/                vo2_3x4.toml, redline.toml, dropout.toml, coast.toml (+  
.zwo)  
├─ docs/  README.md  ENGINEERING_NOTES.md  DEVLOG.md  LATENCY.md  
├─ scripts/ build-ios.sh  build-android.sh  gen-bindings.sh  refresh-ios.sh  
└─ .github/workflows/ci.yml  # fmt, clippy -D warnings, test, no_std check,  
fuzz smoke
```

A single `vc-ffi` cdylib exports both `Engine` and `Simulator`, so there is one bindings pipeline. Feature-gate the simulator (`features = ["sim"]`, on for Android only) to keep the iOS binary small.

(d) Key code skeletons

vc-gatt: CPM decode/encode with round-trip property test

```
rust
```

```

#![no_std]
pub const CPM_PEDAL_BAL: u16 = 1<<0; pub const CPM_TORQUE: u16 = 1<<2;
pub const CPM_WHEEL: u16 = 1<<4; pub const CPM_CRANK: u16 = 1<<5;
// bits 6-11 = further presence bits (extremes, dead spots, energy);
// bits 1, 3 = reference flags, 12 = offset-compensation indicator: NOT present

#[derive(Debug, Clone, Copy, PartialEq, Eq, Default)]
pub struct Cpm { pub power_w: i16, pub crank: Option<(u16, u16)> /* revs, t/102
                pub wheel: Option<(u32, u16)> /* revs, t/2048 */, pub flags: u

#[derive(Debug, PartialEq, Eq)] pub enum ParseError { Short, Reserved }

struct R<'a>(&'a [u8]);
impl<'a> R<'a> {
    fn u16(&mut self) -> Result<u16, ParseError> {
        let (h, t) = self.0.split_first_chunk:::<2>().ok_or(ParseError::Short)?;
        self.0 = t; Ok(u16::from_le_bytes(*h))
    }
    fn skip(&mut self, n: usize) -> Result<(), ParseError> {
        self.0 = self.0.get(n..).ok_or(ParseError::Short)?; Ok(())
    }
    fn u32(&mut self) -> Result<u32, ParseError> {
        let (h, t) = self.0.split_first_chunk:::<4>().ok_or(ParseError::Short)?;
        self.0 = t; Ok(u32::from_le_bytes(*h))
    }
}

pub fn decode_cpm(b: &[u8]) -> Result<Cpm, ParseError> {
    let mut r = R(b);
    let flags = r.u16()?; let power_w = r.u16()? as i16;
    if flags & CPM_PEDAL_BAL != 0 { r.skip(1)?; }
    if flags & CPM_TORQUE != 0 { r.skip(2)?; }
    let wheel = if flags & CPM_WHEEL != 0 { Some((r.u32()?, r.u16()?)) } else {
    let crank = if flags & CPM_CRANK != 0 { Some((r.u16()?, r.u16()?)) } else {
    Ok(Cpm { power_w, crank, wheel, flags }) // later optional fields ignored
}

pub fn encode_cpm(c: &Cpm, out: &mut [u8; 20]) -> usize {
    let mut f = 0u16; let mut n = 4;
    if let Some((rv, t)) = c.wheel { f |= CPM_WHEEL;
        out[n..n+4].copy_from_slice(&rv.to_le_bytes()); out[n+4..n+6].copy_from
    if let Some((rv, t)) = c.crank { f |= CPM_CRANK;
        out[n..n+2].copy_from_slice(&rv.to_le_bytes()); out[n+2..n+4].copy_from
    out[0..2].copy_from_slice(&f.to_le_bytes()); out[2..4].copy_from_slice(&c.p

```

```

}

/// wrap-safe delta for uint16 counters/event times
#[inline] pub fn d16(now: u16, prev: u16) -> u16 { now.wrapping_sub(prev) }

#[cfg(test)] mod t { use super::*; use proptest::prelude::*;
    proptest! {
        #[test] fn roundtrip(p in any::<i16>(), c in proptest::option::of(any::<(u1
            w in proptest::option::of(any::<(u32,u16)>())) {
            let x = Cpm{power_w:p, crank:c, wheel:w, flags:0}; let mut buf=[0u8;20]
            let n = encode_cpm(&x,&mut buf); let y = decode_cpm(&buf[..n]).unwrap()
            prop_assert_eq!((y.power_w,y.crank,y.wheel),(x.power_w,x.crank,x.wheel)
        }
        #[test] fn never_panics(b in proptest::collection::vec(any::<u8>(),0..32))
    }
}

```

Cadence from crank data is $\text{rpm} = \Delta \text{revs} \cdot 60 \cdot 1024 / \Delta t$, using `d16` for both deltas. If $\Delta \text{revs} = 0$ for more than 2 s, cadence is 0 (coasting). Wheel event time is in 1/2048 s units, while CSC (0x2A5B) uses 1/1024 s for both; keep separate constants with tests.

vc-sim: rider step function

```
rust
```

```

pub struct Rider { cp: f32, wp_j: f32, wbal_true: f32, hr: f32, hr_rest: f32, h
k_bpm_per_w: f32, kf_drift_per_kj: f32, tau_on: f32, tau_off: f32,
energy_kj: f32, p_hist: [f32; 16], idx: usize,          // 15 s dead-time b
crank_revs_f: f64, crank_revs: u16, crank_evt_1024: u16, rng: rand_pcg::Pcg

impl Rider {
    /// 1 s physiology step; a separate 4 Hz packetizer produces emissions
    pub fn step(&mut self, t_s: f64, target_w: f32, cad_target: f32, faults: &F
        let cap = if self.wbal_true <= 0.0 { self.cp * 0.92 } else { f32::MAX };
        let noise = 1.0 + 0.03 * normal(&mut self.rng);
        let p = if faults.coasting { 0.0 } else { (target_w.min(cap) * noise).m
        self.wbal_true = if p > self.cp { self.wbal_true - (p - self.cp) }
                        else { self.wbal_true + (self.cp - p) * (self.wp_j - s
        self.energy_kj += p / 1000.0;
        self.p_hist[self.idx % 16] = p; self.idx += 1;
        let p_lag = self.p_hist[(self.idx + 1) % 16];          // ≈15 s ago
        let p_eff = p_lag * (1.0 + self.kf_drift_per_kj * self.energy_kj);
        let hr_ss = (self.hr_rest + self.k_bpm_per_w * p_eff).min(self.hr_max);
        let tau = if hr_ss > self.hr { self.tau_on } else { self.tau_off };
        self.hr += (hr_ss - self.hr) / tau;
        if faults.hr_redline { self.hr = (self.hr + 1.5).min(self.hr_max + 2.0)
        let cad = if p == 0.0 { 0.0 } else if self.wbal_true <= 0.0 { cad_targe
        let prev = self.crank_revs_f; self.crank_revs_f += cad as f64 / 60.0;
        if self.crank_revs_f.floor() > prev.floor() {
            let frac = (self.crank_revs_f.floor() - prev) / (self.crank_revs_f
            let t_evt = t_s - 1.0 + frac;          // time of la
            self.crank_revs = self.crank_revs.wrapping_add((self.crank_revs_f.f
            self.crank_evt_1024 = ((t_evt * 1024.0) as u64 & 0xFFFF) as u16;
        } // else unchanged count/time = stale-looking coasting packet
        Truth { p, hr: self.hr, cad, wbal: self.wbal_true }
    }
}

```

vc-physio: differential W'bal (GoldenCheetah default model)

rust

```

pub struct WbalDiff { cp: f32, wp: f32, bal: f32, last_ns: Option<u64> }
impl WbalDiff {
    pub fn update(&mut self, t_ns: u64, p: f32) -> f32 {
        let dt = match self.last_ns.replace(t_ns) {
            Some(prev) => ((t_ns - prev) as f32 * 1e-9).clamp(0.0, 5.0), // gap
            None => return self.bal };
        self.bal = if p > self.cp { self.bal - (p - self.cp) * dt }
                    else { self.bal + (self.cp - p) * dt * (self.wp - self.bal)
        self.bal = self.bal.min(self.wp); // allow negative: "below
        self.bal
    }
}

```

vc-core: arbiter (priority + cooldown + hysteresis + staleness)

```

rust

pub enum Prio { Safety = 0, Structure = 1, Effort = 2, Info = 3 }
pub struct Candidate { kind: CueKind, prio: Prio, made_ns: u64, inputs_fresh: b
pub struct Arbiter { last_spoken_ns: u64, per_kind_last: [u64; N_KINDS], speaki

impl Arbiter {
    pub fn pick(&mut self, now: u64, cands: &mut Vec<Candidate>) -> Option<CueK
        cands.retain(|c| c.inputs_fresh && now - c.made_ns < 2_000_000_000); /
        cands.sort_by_key(|c| c.prio as u8);
        for c in cands.iter() {
            let safety = matches!(c.prio, Prio::Safety);
            if !safety && now < self.speaking_until { return None; }
            if !safety && now - self.last_spoken_ns < 8_000_000_000 { return No
            if now - self.per_kind_last[c.kind as usize] < c.kind.cooldown_ns()
            self.per_kind_last[c.kind as usize] = now; self.last_spoken_ns = now
            self.speaking_until = now + c.kind.duration_ns();
            return Some(c.kind);
        }
        None
    }
}

// Hysteresis in triggers: "Ease off" at W'bal < 25 %, re-arm > 35 %.

```

vc-ffi: UniFFI Engine object

```

rust

```

```

uniffi::setup_scaffolding!();

#[derive(uniffi::Enum)] pub enum Source { Hrm, Cpm, Csc, Telemetry }
#[derive(uniffi::Record)] pub struct CueOut { pub id: u32, pub phrase_key: Stri
#[derive(uniffi::Record)] pub struct Dash { pub power: i32, pub hr: i32, pub ca
    pub step_label: String, pub hr_state: String, pub last_cue: Option<String>

#[derive(uniffi::Object)] pub struct Engine(std::sync::Mutex<vc_core::Engine>);

#[uniffi::export]
impl Engine {
    #[uniffi::constructor] pub fn new(cfg_toml: String) -> Result<std::sync::Ar
    /// zero-copy: Swift passes Data; Kotlin (0.32+) would pass a direct ByteBu
    pub fn ingest(&self, t_ns: u64, src: Source, payload: &[u8]) -> bool {
        self.0.lock().unwrap().ingest(t_ns, src.into(), payload).is_ok()
    }
    pub fn tick(&self, t_ns: u64) -> Vec<CueOut> { self.0.lock().unwrap().tick(
    pub fn mark_audio_submitted(&self, cue_id: u32, t3_ns: u64, est_out_latency
    pub fn snapshot(&self) -> Dash { /* copy-out */ }
    pub fn latency_csv(&self) -> String { /* p50/p99 per segment */ }
}

```

Kotlin: GATT server notify loop (one in flight)

kotlin

```

class SimService : Service() {
    private val sim = Simulator(scenarioToml, seed = 42u) // Rust via
    private val q = ArrayDeque<Pair<BluetoothGattCharacteristic, ByteArray>>()
    @Volatile private var inFlight = false
    private lateinit var server: BluetoothGattServer
    private val subscribers = mutableSetOf<BluetoothDevice>() // CCCD 0x29
    private val h = Handler(HandlerThread("sim-tick").apply { start() }.looper)

    private val tick = object : Runnable { override fun run() {
        for (e in sim.step(SystemClock.elapsedRealtimeNanos().toULong()))
            q.add(charFor(e.channel) to e.bytes) // bytes al
        pump(); h.postDelayed(this, 250) } }

    private fun pump() {
        if (inFlight || q.isEmpty()) return
        val dev = subscribers.firstOrNull() ?: run { q.clear(); return }
        val (ch, bytes) = q.removeFirst()
        val rc = server.notifyCharacteristicChanged(dev, ch, false, bytes) //
        inFlight = rc == BluetoothStatusCodes.SUCCESS
        if (!inFlight) h.postDelayed({ pump() }, 10)
    }

    private val cb = object : BluetoothGattServerCallback() {
        override fun onNotificationSent(d: BluetoothDevice, status: Int) { h.po
        override fun onDescriptorWriteRequest(d: BluetoothDevice, id: Int, desc
            prep: Boolean, resp: Boolean, off: Int, v: ByteArray) {
            if (v.contentEquals(BluetoothGattDescriptor.ENABLE_NOTIFICATION_VAL
            if (resp) server.sendResponse(d, id, BluetoothGatt.GATT_SUCCESS, 0,
            // onCharacteristicWriteRequest: CTRL (scenario, fault, PING) → sim.con
        }

        // onCreate: startForeground(..., FOREGROUND_SERVICE_TYPE_CONNECTED_DEVICE)
        // (await onServiceAdded), THEN startAdvertising
    }
}

```

Swift: CoreBluetooth delegate → Rust

swift

```

final class BLE: NSObject, CBCentralManagerDelegate, CBPeripheralDelegate {
    private let q = DispatchQueue(label: "ble.serial", qos: .userInteractive)
    private lazy var cm = CBCentralManager(delegate: self, queue: q)
    let engine: Engine // UniFFI object, Sendable
    private static let svcs = [CBUUID(string: "180D"), CBUUID(string: "1818"),

    func centralManagerDidUpdateState(_ c: CBCentralManager) {
        if c.state == .poweredOn { c.scanForPeripherals(withServices: Self.svcs
    func centralManager(_ c: CBCentralManager, didDiscover p: CBPeripheral, adv
        guard (advertisementData[CBAvertisementDataLocalNameKey] as? String) =
            peripheral = p; p.delegate = self; c.stopScan(); c.connect(p) }
    func peripheral(_ p: CBPeripheral, didUpdateValueFor ch: CBCharacteristic,
        let t1 = clock_gettime_nsec_np(CLOCK_UPTIME_RAW) // stamp FIRS
        guard error == nil, let v = ch.value else { return }
        _ = engine.ingest(tNs: t1, src: Self.source(for: ch.uuid), payload: v)
        // didDisconnect → engine marks channels Lost; reconnect with backoff; redi
    }
    // Cue timer on `q` @4 Hz: for cue in engine.tick(now) { audio.play(cue) } → en

```

(e) Environment checklist for the 8 GB M2

bash

```

df -h / # free ≥ 60 GB first
# 1. Xcode (free Apple ID); skip extra simulator runtimes
xcode-select --install || true
sudo xcodebuild -license accept
xcodebuild -downloadPlatform iOS
# 2. Homebrew + tools
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/H
brew install --cask temurin android-commandlinetools
brew install gradle python@3.12 jq
# 3. Android SDK/NDK without Android Studio
export ANDROID_HOME="$(brew --prefix)/share/android-commandlinetools"
yes | sdkmanager --licenses
sdkmanager "platform-tools" "platforms;android-36" "build-tools;36.0.0" "ndk;30
export ANDROID_NDK_HOME="$ANDROID_HOME/ndk/30.0.16248370"
# 4. Rust
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
rustup target add aarch64-apple-ios aarch64-apple-ios-sim aarch64-linux-android
cargo install cargo-ndk cargo-nextest cargo-fuzz # fuzz runs need nightly
# 5. Piper (offline phrase bank)
python3 -m venv ~/.venvs/piper && source ~/.venvs/piper/bin/activate
pip install piper-tts && python -m piper.download_voices en_US-ljspeech-medium
# 6. ~/.cargo/config.toml: [build] jobs = 4 ; [profile.dev] debug = "line-table
# [profile.dev.package.*] opt-level = 1
# 7. ~/.gradle/gradle.properties: org.gradle.jvmargs=-Xmx1536m org.gradle.daem

```

Build commands

```

bash

# iOS XCFramework (device slice; add aarch64-apple-ios-sim only if you ever use
cargo build -p vc-ffi --release --target aarch64-apple-ios
cargo run -p uniffi-bindgen -- generate --library target/aarch64-apple-ios/rele
--language swift --out-dir apps/ios/Generated
xcodebuild -create-xcframework \
  -library target/aarch64-apple-ios/release/libvc_ffi.a -headers apps/ios/Gene
  -output apps/ios/VcCore.xcframework
# Android
cargo ndk -t arm64-v8a -o apps/android/vcsim/app/src/main/jniLibs build -p vc-f
cargo run -p uniffi-bindgen -- generate --library target/aarch64-linux-android/
--language kotlin --out-dir apps/android/vcsim/app/src/main/java
cd apps/android/vcsim && gradle assembleDebug && adb install -r app/build/outpu

```

The `uniffi-bindgen` binary is a tiny workspace crate that depends on `uniffi = { version = "=0.32.2", features = ["cli"] }` and calls

`uniffi::uniffi_bindgen_main()`. `sal` `github` The headers directory needs the generated `.h` plus a `module.modulemap`.

RAM, disk and provisioning habits. Never run Xcode and Gradle builds at the same time, close Chrome while building, and deploy to the real iPhone rather than the Simulator. Clear `~/Library/Developer/Xcode/DerivedData/*` every few days, run `xcrun simctl delete unavailable`, and prune `target/` weekly. Use one bundle ID (e.g. `dev.zakir.velocoach`), re-run from Xcode every Monday, keep a `refresh-ios.sh` (`xcodebuild` + `xcrun devicectl device install app`), and **re-sign the day before the trial and before recording**.

(f) 20-day schedule (7 h/day)

✅ marks a checkpoint you must hit.

Week 1: foundations and a radio smoke test

- **D1:** Env setup (3 h; start the Xcode download first). Radio smoke test (2 h): nRF Connect on the iQOO with the Auuki HRM/power configs, plus a 30-line SwiftUI scanner on the iPhone (free team, Developer Mode). Workspace skeleton and CI (2 h).
✅ The iPhone displays HR notifications from the iQOO.
- **D2:** vc-gatt HRM decode/encode plus tests (3 h). CPM decode/encode, presence bits and `d16` (3 h). Check the Intervals.icu API key with curl (1 h).
- **D3:** CSC decode/encode (2 h). Proptest round-trips, never-panic suite, cargo-fuzz targets (3 h). `#![no_std]` CI check via a `thumbv7em-none-eabihf` build (1 h). README skeleton (1 h). ✅ vc-gatt done and fuzzed.
- **D4:** vc-physio differential W'bal plus analytic tests (3 h). Waterworth integral variant (2 h). Data-quality state machine (2 h).
- **D5:** Decoupling (rolling Pa:HR with warm-up exclusion) plus tests (3 h). vc-sim rider model: HR kinetics, W' truth, blow-up (4 h).
- **D6:** Crank-event generator with wrap tests: 3 hours of sim time, decoded cadence = truth ± 1 rpm (3 h). Seeded noise, scenario TOML, fault scheduler (4 h). ✅ `vc-cli scenario vo2_3x4 --seed 42` prints truth vs decoded.
- **D7:** vc-core timeline (WorkoutPlan steps), SafetyGate, Arbiter (5 h). Cue phrase rules and a syllable-counting test (2 h).

Week 2: end-to-end across three devices

- **D8:** vc-ffi Engine + Simulator; Swift XCFramework script, link in a bare iOS app, call `snapshot()` (5 h). Kotlin bindings, JNA aar, `Simulator.step()` logging on Android (2 h). ✅ Rust running on both phones.
- **D9:** Android GATT server: services, advertising/scan response, CCCD handling, notify queue, FGS, Vivo settings (6 h). Test with nRF Connect as the client (1 h). ✅ Rust-generated packets visible in a third-party client.

- **D10:** iOS central: scan/connect/discover/subscribe, `ingest` on the serial queue, 4 Hz tick, minimal SwiftUI dashboard (6 h). Reconnect/backoff (1 h).
- **D11:** phrasagen: Piper bank of 40–60 phrases plus manifest (2 h). iOS AVAudioEngine player plus `markAudioSubmitted` (3 h). First full VO2 run over radio with spoken cues (2 h).  **End-to-end demo exists.** Tag `v0.1`.
- **D12:** Telemetry characteristic, PING/PONG clock sync, latency ring, CSV export (6 h). vc-cli `latency` report, p50/p99 per segment (1 h).
- **D13:** vc-cli `live` (btleplug against the iQOO) and `replay` (CSV/FIT via fitparser) (5 h). Golden cue-log tests for every scenario in CI (2 h).
- **D14:** Fault scenarios end to end: dropout, garbage, disconnect, coasting silent/stale, HR redline, W' mismatch (5 h). Fix what breaks (2 h).  Every scenario passes on radio and in CI.

Week 3: harden, measure, present

- **D15:** vc-sync.zwo parser plus Intervals.icu fetch in vc-cli, driving sim and coach from the same plan (5 h). Optional iOS API-key entry (2 h, cuttable).
- **D16:** criterion benchmarks for `ingest`/`tick`, allocation counting, 1-hour radio soak (iQOO screen on, plugged in), record drops/reconnects (7 h).
- **D17:** Latency dataset: 3×20 -minute runs, earbuds and speaker routes; fill in LATENCY.md (4 h). ENGINEERING_NOTES (3 h).
- **D18:** Dashboard polish, cue log view, app icon, DEVLOG cleanup (4 h). Rehearse the video script (3 h).
- **D19:** Record and edit the video in iMovie (5 h). README final pass (2 h).
- **D20:** Buffer/fixes (3 h). Re-sign the iOS app; send the message to Aki with repo and video links (1 h). Practise the explanations in (k) (3 h).

Cut list (in this order): FTMS service → Waterworth variant → iOS-side Intervals.icu → dual-central test → CSC service (CPM crank data already gives cadence) → btleplug live mode (keep replay) → fuzzing beyond a smoke run. **Never cut:** round-trip tests, the safety gate, staleness handling, the latency table, or the golden cue logs.

(g) Demo scenarios and the 3-minute storyboard

Scenarios, each a seeded TOML that mirrors a .zwo:

1. `vo2_3x4`: 5-minute warm-up, then 3×4 min at 120% CP with 3 min recovery. W' is set so rep 3 runs it to about 5% ("Hold it steady, forty seconds left"-type cues, at most one number each).
2. `blowup`: true W' 30% smaller than the coach's estimate. The rider fades mid-rep 2, power collapses to CP and cadence sags; the coach issues "Ease to tempo now" and re-anchors.
3. `redline`: an HR ramp to HR_max triggers the SafetyGate, which pre-empts a structure cue: "Stop the effort, spin easy."

4. **dropout**: HR silent for 20 s → Stale → HR-based cues suppressed and one "Heart rate signal lost" cue; recovery after 3 good samples.
5. **coast**: 15 s of stale crank data, then silence. Cadence goes to 0, no false "Pedal faster" cue, and W' recovers.
6. **garbage**: 50 malformed packets. Parse-error counters increase and nothing crashes.

Storyboard (3:00)

Time	Shot	Voice-over / on-screen text
0:00–0:15	Three devices on the desk; title card	"velocoach-rs: one Rust core, real BLE radio, simulated rider."
0:15–0:35	Architecture diagram	Android simulator encodes SIG packets in Rust → iPhone decodes in Rust → voice cue.
0:35–1:20	iPhone recording (speaker audio) during vo2_3x4 rep 3; iQOO picture-in-picture	W' draining, cues spoken, cue log ticking.
1:20–1:45	redline	SafetyGate overrides the interval cue; "SAFETY" badge.
1:45–2:10	dropout and coast	HR goes Stale, cues suppressed, recovery; no false cadence nag.
2:10–2:35	Mac terminal	cargo test , vc-cli scenario --seed 42 twice with identical logs, then the latency table.
2:35–3:00	README latency table plus honest note	"Sensor→decision p50/p99 measured; audio estimate shown separately; A2DP adds ~160 ms." End card with repo link.

(h) README, latency table, ENGINEERING_NOTES, framing

README structure: what this is (2 sentences plus the framing below); 3-minute video; architecture diagram; quick start (**cargo test**, **vc-cli scenario**, phone builds); scenarios; latency results; correctness (tests, fuzzing, golden logs); limitations; licences (data, voice, deps).

Latency table template

Segment	Clock	n	p50 (ms)	p99 (ms)	max	Notes
T0→T1 notify→ingest (radio + stacks)	cross-device					$\pm \text{rtt_min}/2 = __ \text{ ms}$
T1→T2 ingest→decision	iPhone					Rust core only
T2→T3 decision→audio submit	iPhone					AVAudioEngine schedule
T0→T3 sensor→audio submit	mixed					the "<150 ms" claim
T3→T4 output-latency estimate (earbuds, A2DP)	estimate					typically ~160 ms; not measured acoustically

ENGINEERING_NOTES outline: (1) why sans-IO; (2) GATT parsing pitfalls: presence bits, 1/2048 vs 1/1024, wraparound, coasting staleness; (3) W'bal model choice and validation; (4) decoupling and warm-up; (5) arbiter design; (6) SafetyGate invariants; (7) clock sync and its error bound; (8) platform notes: iOS GATT caching, Android notify queue, Vivo FGS; (9) what changes for real sensors; (10) open questions for Domesly.

Framing language: "This is hardware-in-the-loop over a real BLE link with simulated sensors. The iQOO emits byte-exact Bluetooth SIG Heart Rate, Cycling Power and CSC notifications produced by a deterministic rider model. Nothing in the coaching path knows it is a simulation. The simulator lets me reproduce faults (sensor loss, coasting, garbage packets, W' exhaustion, HR redline) that I can't create safely or repeatably on a ride. Real-sensor validation is the next step and needs only a strap and a power meter."

(i) Message to Aki

Hi Aki — ahead of the Rust trial I built a small vertical slice of a real-time cycling voice coach to show how I'd approach the core engine: **velocoach-rs** (repo + 3-min video: <links>).

One sans-IO Rust core runs on both phones via UniFFI: an Android phone simulates a rider and emits byte-exact BLE Heart Rate/Cycling Power/CSC notifications; an iPhone decodes them, computes W'bal and decoupling, and speaks short cues through a priority arbiter with a deterministic safety gate. Parsers are no_std, panic-free and fuzzed; every scenario (W' blow-up, HR redline, dropout, coasting) is seeded and golden-tested. Sensor→audio-submit p50/p99 is in the README, with Bluetooth audio latency reported separately and honestly.

It's simulation over real radio, not a ride — I'd love to hear where it diverges from what you're seeing with real sensors. Looking forward to the trial. — Zakir

(j) Risks and fallbacks

Risk	Likelihood	Mitigation / fallback
iQOO kills the FGS or advertising stops in background	Medium-high	Foreground app, screen on, locked in recents, "High background power consumption" + Autostart. <code>(dontkillmyapp)</code> <code>(biolovision)</code> Fallback: nRF Connect plus Auuki configs (fixed sample data).
<code>(getBluetoothLeAdvertiser())</code> null or advertising fails	Low	Check on D1. Fallback: Mac as peripheral via a small Swift CBPeripheralManager tool (verify HRS/CPS aren't blocked) with the iPhone as central. Last resort: FIT/CSV replay only.
iOS sees "no services" after simulator restart	Medium	Never mutate services while connected; cancel and rediscover on iOS; toggle iPhone Bluetooth if stuck. <code>(appspot)</code>
Free profile expires before recording or mid-trial	Certain every 7 days	Re-sign D19 and the day before the trial; keep <code>(refresh-ios.sh)</code> ready.
Xcode 27 needs a newer macOS than installed	Medium	Upgrade macOS on D1 (free). You can't downgrade iOS.
8 GB RAM swapping	High during builds	One platform at a time, jobs = 4, no simulators, small debug info.
UniFFI 0.32 Kotlin bytes break	Medium	Use <code>(Vec<u8>)</code> (ByteArray) for the Android-facing API, which is off the hot path; <code>(&[u8])</code> for iOS.

(k) Be ready to explain in the trial

- Why sans-IO and synchronous: testability, determinism, no async runtime on mobile, and how the shells own threading.
- CPM flag layout and the presence vs non-presence bits; wrap-safe deltas; why coasting looks like a repeated event time and then silence.
- Differential vs integral W'bal: the implicit $\tau = W'/CP$, gap handling, negative balances, and validation against GoldenCheetah outputs.
- How the arbiter avoids talk-over, cue spam and stale advice, and how tests prove the SafetyGate pre-empts.
- What sub-150 ms means here, segment by segment: which segments use one clock and which use cross-device sync with an error bound. Why A2DP adds ~160 ms on top

and what Domesly could do: pre-roll an audio session, keep the route warm, use short pre-rendered clips.

- The UniFFI 0.32 bytes change, Mutex-in-Object vs actor design, and how you'd move to `Cxx` if they need C++ interop.

Caveats

- No Apple source explicitly says CoreBluetooth background modes work on free personal teams; test it on day 1. Xcode 27's exact macOS minimum is also unconfirmed.
- The GoldenCheetah OpenData licence is unconfirmed, so don't redistribute those files.
- HR model parameters come from small studies, and the ~160 ms `outputLatency` figures come from AirPods reports, not your earbuds. Treat both as typical, not guaranteed.